



A

PROJECT REPORT

FOR

SUBJECT: PROJECT PHASE II

ON

Schedulo- Automated Timetable Generator

Submitted in partial fulfilment of the requirement for the award of

Bachelor of Technology

In

Computer Science and Engineering

**Walchand Institute of Technology
(An Autonomous Institute)**

By

Name	Roll. No.	Exam Seat No.
Aditi Deshpande	05	2204111121
Priyanka Sundalam	06	2302112014
Ankita Advitot	07	2302112011
Tejas Jahagirdar	08	2202111175

Under the Guidance of

Dr. Mrs. A. M. Pujar

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
WALCHAND INSTITUTE OF TECHNOLOGY
SOLAPUR - 413006
(2025-2026)**



Certificate

This is to certify that the project entitled

Schedulo- Automated Timetable Generator

Is submitted by

Name	Roll. No.	Exam Seat No.
Aditi Deshpande	05	2204111121
Priyanka Sundalam	06	2302112014
Ankita Advitot	07	2302112011
Tejas Jahagirdar	08	2202111175

(Dr. Mrs. A. M. Pujar)
Project Guide

(Dr. Mrs. A. M. Pujar)
Head
Dept of Computer Sc. & Engg

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
WALCHAND INSTITUTE OF TECHNOLOGY
SOLAPUR
(2025-2026)**

Project Approval Sheet

The Project Entitled

Schedulo – Automated Timetable Generator

Submitted by

Name	Roll No.	Exam Seat No.
Aditi Deshpande	05	2204111121
Priyanka Sundalam	06	2302112014
Ankita Advitot	07	2302112011
Tejas Jahagirdar	08	2202111175

"Is hereby approved in partial fulfilment for the degree of Bachelor of Computer Science and Engineering"

(Dr. Mrs. A. M. Pujar)
Project Guide

(Dr. Mrs. A. M. Pujar)
Head
Dept of Computer Sc. & Engg

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
WALCHAND INSTITUTE OF TECHNOLOGY, SOLAPUR – 413006
(2025–26)

Acknowledgment

At the outset, we would like to take this opportunity to express our deep gratitude to our guide Dr. A. M. Pujar of the CSE Department for her invaluable guidance, constant encouragement, and moral support throughout the successful completion of our project.

We heartily thank Dr. Mrs. A. M. Pujar, Head of CSE Department, for her moral support and for promoting us through the completion of our project.

We would also like to thank our Principal, Dr. V. A. Athavale, and all the staff members of WIT Solapur for their wholehearted co-operation and continued encouragement in completing this project.

Finally, we extend our gratitude to our families and peers whose constant motivation helped us persevere through the challenges encountered during this project.

Undertaking

We solemnly declare that the project work presented in the report titled **Schedulo – Automated Timetable Generator** is solely our project work with no significant contribution from any other person except our project guide. Small contributions or help wherever taken have been duly acknowledged, and the complete report has been written by the members of the project group.

We understand the zero tolerance policy of WIT, Solapur towards plagiarism. Therefore, we as authors of the above titled report declare that no portion of the report has been plagiarized and any material used as reference is properly referred/cited.

We undertake that if found guilty of any formal plagiarism in the above titled report even after award of the degree, WIT Solapur and PAHSUS reserve the right to withdraw/revoke the degree granted, and that WIT Solapur and the University have the right to publish our names on the website on which names of students are placed who submitted plagiarized reports.

Name	Exam Number	University PRN Number	Signature
Aditi Deshpande		2204111121	
Priyanka Sundalam		2302112014	
Ankita Advitot		2302112011	
Tejas Jahagirdar		2202111175	

Date: ____ / ____ / ____

Abstract

The project "**Schedulo – Automated Timetable Generator**" aims to fully automate the process of academic timetable generation for engineering institutions, eliminating the manual effort, errors, and conflicts typically associated with the scheduling process.

Problem: Manual timetable creation is error-prone, time-consuming, and frequently results in conflicts such as double-booked faculty, batch overlaps, underutilized laboratories, and unbalanced subject distribution across the week.

Method: In Phase II, the system has been upgraded with a three-stage algorithmic pipeline: (1) a Practical Generator using Greedy CSP logic with a Round-Robin scheduling order ($SY \rightarrow SY \rightarrow TY \rightarrow TY \rightarrow BE$), (2) a Class Timetable Handler with a Lab-to-Class Data Mapper, and (3) a Smart Queue-Based Lecture Generator with a two-tier morning/afternoon slot placement strategy and in-memory constraint caching. The system additionally implements JWT-based Authentication and Role-Based Authorization (Admin/Faculty), a Special Constraints module (preferred_off and fixed_time), and CSV export for all timetable views.

Results: The system successfully generates complete, conflict-free timetables for practicals and lectures across all divisions (SY/TY/BE). Timetables are viewable per class division, per faculty, and per laboratory, and are exportable as CSV files.

Conclusion: Schedulo Phase II delivers a fully automated, constraint-aware, secure, and end-to-end timetable generation platform. Future work will explore meta-heuristic optimization, dynamic rescheduling, and cloud deployment.

Index

Sr. No.	Title	Page No.
1	Introduction	8
	1.1 Introduction	8
	1.2 Problem Statement and Objective	9
2	Background	10
3	Proposed Solution	13
	3.1 Solution	13
	3.2 Advantages of Proposed System	17
4	Working Environment	18
	4.1 Hardware Requirements	18
	4.2 Software Requirements	18
5	Methodology	20
	5.1 System Architecture	20
	5.2 Work Flow	21
6	Flow Diagrams	22
	6.1 Data Flow Diagram	22
	6.2 Sequential UML Diagram	23
7	Implementation	24
	7.1 Code Snippet	24
	7.2 Screenshots & Results	27
8	Testing Report	35
9	Conclusion	37
10	Future Work	38
11	References	40

Chapter 1

INTRODUCTION

1.1 Introduction

In educational institutions, designing a well-structured timetable is one of the most complex and time-consuming administrative activities. It requires balancing numerous constraints simultaneously: faculty availability, subject credit requirements, laboratory allocation, student batch management, break configurations, and time slot optimization. Traditionally, this process is done manually, resulting in frequent conflicts such as overlapping sessions, underutilized laboratories, and unbalanced faculty workloads.

The project "**Schedulo – Automated Timetable Generator**" addresses these challenges through a complete, automated scheduling system. Developed across two phases, the system evolved from a Phase I Brute Force prototype into a fully featured Phase II platform with the following capabilities:

- Step 1 – Practical Generator: Greedy CSP algorithm with a Round-Robin scheduling cycle (SY → SY → TY → TY → BE) replacing the previous fixed-priority approach, producing a Master Lab Timetable.
- Step 2 – Class Timetable Handler: Maps lab-wise data to class-wise format, locking practical slots per division.
- Step 3 – Lecture Generator: Smart Queue-Based system with in-memory constraint caching and a two-tier morning/afternoon slot placement strategy.
- Authentication & Authorization: JWT-based login with role-based access control separating Admin and Faculty capabilities.
- Special Constraints Module: Faculty can declare preferred-off times; Admins can pin fixed lecture times for specific subjects.
- CSV Export: All timetable views (Class, Faculty, Lab, Master Practical) can be downloaded as CSV files.

The final output is a fully automated, conflict-free timetable for all academic years (SY/TY/BE) across all divisions, viewable per class, per faculty, and per laboratory.

1.2 Problem Statement and Objective

Problem Statement: Educational institutions face significant difficulty in manually creating academic timetables that satisfy all institutional constraints simultaneously. The process is error-prone, time-consuming, and frequently results in conflicts such as double-booked faculty, overlapping practicals for the same batch, underutilized laboratory resources, and unbalanced subject distribution. Existing commercial tools offer limited customization and do not cater to the specific departmental structures and batch-division configurations of engineering colleges in India.

Objectives:

- To design and implement a fully automated, constraint-aware timetable generation system for practical and lecture sessions in engineering institutions.
- To implement a Greedy CSP-based Practical Generator using a Round-Robin scheduling cycle ($SY \rightarrow SY \rightarrow TY \rightarrow TY \rightarrow BE$) to ensure fair and conflict-free batch allocation.
- To develop a Smart Queue-Based Lecture Generator with in-memory constraint caching and a two-tier (Morning/Afternoon) fallback strategy for maximum slot utilization.
- To implement JWT-based Authentication and Role-Based Authorization distinguishing Admin and Faculty roles.
- To provide a Special Constraints module allowing faculty to register preferred-off time slots and admins to pin fixed lecture times for specific subjects/divisions.
- To generate all timetable outputs as downloadable CSV files for Class Timetables, Faculty Timetables, Lab Timetables, and the Master Practical Plan.
- To build a flexible, modular, and scalable architecture that supports future enhancements such as meta-heuristic optimization and dynamic rescheduling.

Chapter 2

BACKGROUND

Timetable generation is a well-studied NP-hard combinatorial optimization problem in computer science and operations research. The following literature review traces the evolution of scheduling approaches from manual methods to modern algorithmic and AI-based systems.

[1] ASC Timetables (n.d.)

ASC Timetables is a commercial scheduling tool widely used in schools. While it provides a graphical constraint-entry interface, it lacks full automation and cannot accommodate the multi-batch, multi-division practical structures of engineering colleges in India.

[2] TimetableMaster (n.d.)

TimetableMaster provides semi-automated scheduling primarily for secondary schools. It requires significant manual intervention for complex constraint configurations and does not support multi-year, multi-division engineering department structures.

[3] Alonge, O., & Sakpere, W. (2024)

This study applies the Non-dominated Sorting Genetic Algorithm II (NSGA-II) to timetable scheduling, balancing multiple conflicting objectives simultaneously. While near-optimal results are achieved, the approach is computationally expensive for real-time institutional use and requires significant parameter tuning.

[4] Farinola, L., & Assogba, M. (2025)

This paper presents an AI-based timetable generator for colleges using constraint logic programming. It achieves high accuracy in conflict-free schedule generation and serves as a direct reference for the CSP-based Practical Generator adopted in Schedulo.

[5] Russell, S., & Norvig, P. – Artificial Intelligence: A Modern Approach

The foundational CSP framework from this textbook underpins Schedulo's Practical Generator. Variables (time slots), domains (faculty/lab combinations), and constraints (no double-booking) are formally defined following this reference.

[6] Colorni, A., Dorigo, M., & Maniezzo, V. (1992)

One of the earliest papers to apply Genetic Algorithms to the timetabling problem. This work provides the theoretical foundation for the meta-heuristic optimization layer planned for Schedulo Phase III.

[7] Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983) – Simulated Annealing

The seminal Simulated Annealing paper. SA is planned as a post-processing optimization layer in Schedulo to improve timetable quality after initial generation by escaping local optima.

[8] Glover, F. (1989) – Tabu Search

Tabu Search uses memory structures to avoid revisiting recent solutions. This technique is considered for integration in Schedulo to further refine timetables by iteratively swapping slots to reduce soft constraint violations.

[9] Google OR-Tools (n.d.)

Google OR-Tools provides constraint programming and combinatorial optimization solvers. The CP-SAT solver is being evaluated for potential integration in Schedulo to handle large-scale scheduling instances more efficiently than the current Greedy approach.

[10] OWASP – JSON Web Token (JWT) Best Practices

Schedulo's authentication layer is built following JWT best practices as documented by OWASP. HS256-signed tokens with 7-day expiry are used for session management, with role claims (admin/faculty) embedded in the payload for role-based access control.

[11] MongoDB Documentation (MongoDB, Inc., n.d.)

MongoDB is the NoSQL database used in Schedulo for storing faculty records, subject configurations, lab data, class structures, constraints, settings, and generated timetable outputs. Its flexible document schema suits the hierarchical and dynamic nature of academic scheduling data.

Chapter 3

PROPOSED SOLUTION

3.1 Solution

Schedulo addresses the academic timetabling problem through a structured three-stage algorithmic pipeline integrated with a secure web application. The system features a React.js frontend, a Python Flask backend with JWT-based authentication, and a MongoDB database.

Project Workflow – Input Structure

The system accepts input data in two structured steps before scheduling begins:

Step 1 – Master Data Entry: Three categories of data are collected: Faculty Data (title, full name, short name, email), Labs Data (lab name and short form, lab session timings), and Class Structure (number of divisions and batches per division for SY/TY/BE).

Step 2 – Configuration: Three additional modules are configured: Department Timings (lecture duration, day start/end time, repeatable break definitions), Subject Entry (year, full/short name, lectures/week, practicals/week, practical duration and type, required lab), and Faculty Assignment (year, subject, faculty, division, batches if practical, theory hrs/week, practical hrs/week).

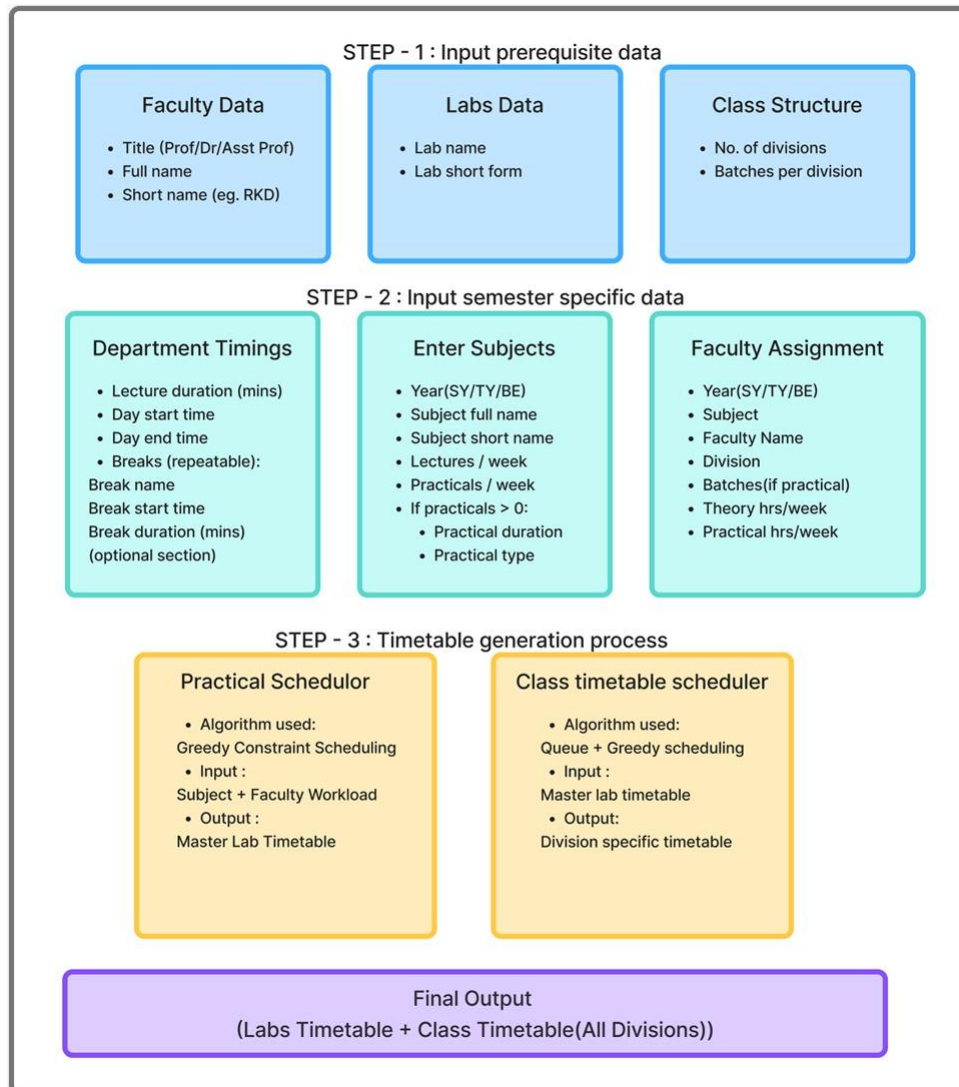


Fig. 1: Scheduling Project Workflow and Input Data Structure

Timetable Generator Algorithm – Three-Step Pipeline

Step 1 – Practical Generator (Greedy CSP + Round-Robin): The system loads all constraints into memory at startup (TG-CACHE FIX). It then applies a Greedy CSP algorithm using a weighted Round-Robin scheduling cycle (SY → SY → TY → TY → BE, repeating) instead of a fixed priority queue. This ensures BE is visited once every five turns and is never locked out of slot assignment. Hard constraints enforced include: no faculty double-booking, no lab double-booking, no batch double-booking, and duplicate workload prevention. The output is a Master Lab Timetable.

Step 2 – Class Timetable Handler: The Master Lab Timetable is passed through a Data Mapper that converts lab-wise scheduling data into class-wise format (batch integer normalization applied), producing a Locked Practical schedule per division.

Step 3 – Lecture Generator (Smart Queue + In-Memory Constraints): Using a Smart Queue-Based system with Round-Robin ordering, the generator auto-detects available lecture slots. All constraints (preferred_off and fixed_time) are loaded into memory once at startup (LG-CACHE FIX) to prevent per-slot database queries. Fixed-time constraints are applied first (Phase 0) before the main scheduling loop. The system operates in Tier 1 (Morning) mode by default, falling back to Tier 2 (Afternoon) if the faculty is busy. The spread constraint prevents the same subject from appearing twice on the same day for the same class.

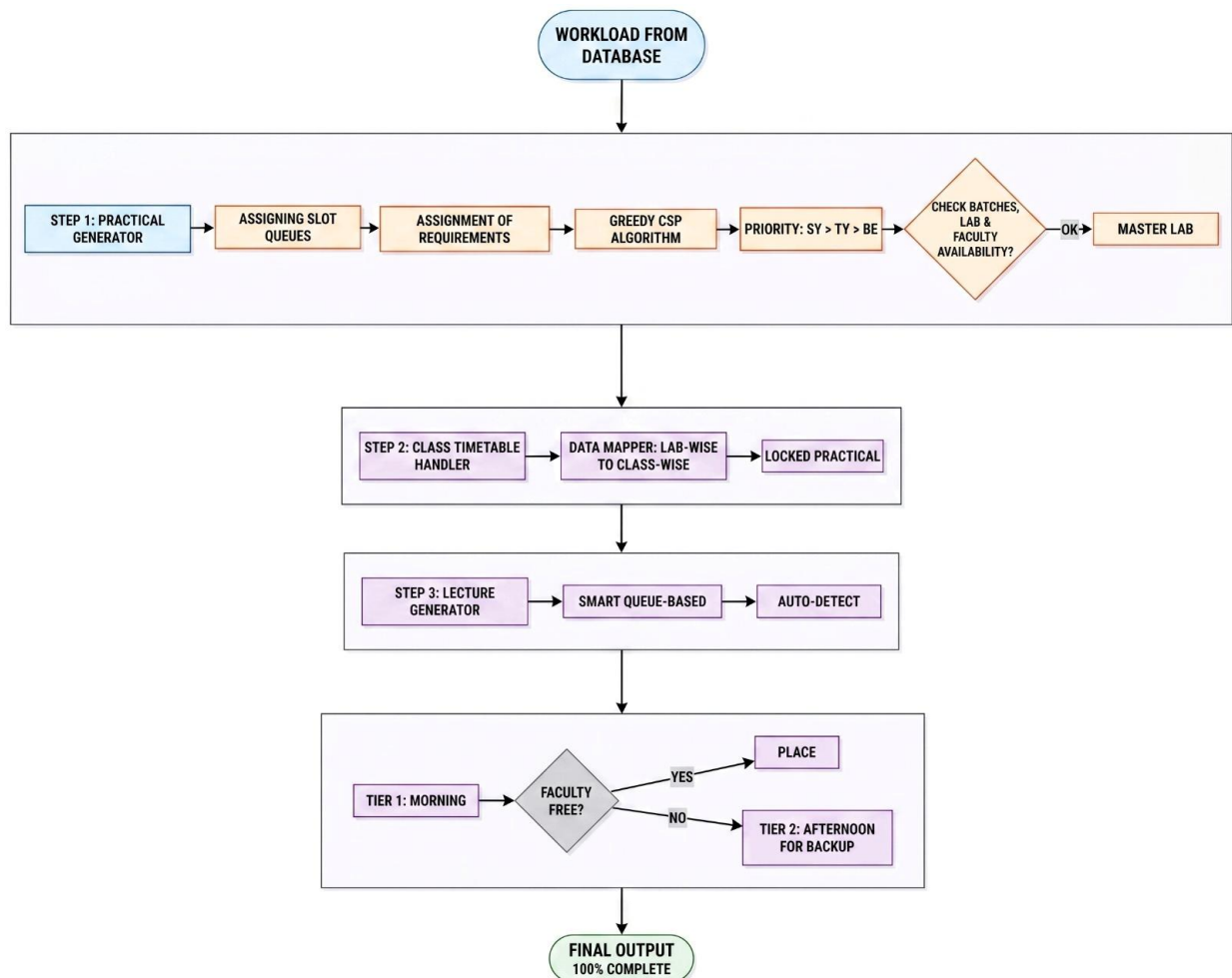


Fig. 2: Timetable Generator Algorithm – Three-Step Pipeline

Authentication & Authorization

The system implements JWT-based Authentication with Role-Based Access Control (RBAC). When a faculty member logs in, the Flask backend validates credentials against hashed passwords stored in MongoDB (using Werkzeug password hashing). A signed JWT token (HS256, 7-day expiry) is issued containing the user ID, name, email, and role (admin or faculty). The React frontend stores this token in localStorage and attaches it as a Bearer token in all subsequent API requests via an Axios interceptor.

Two Flask decorators enforce security on every protected route: `@token_required` validates the JWT and attaches the decoded user to the request context; `@role_required('admin')` restricts sensitive operations (generating timetables, managing faculty, labs, subjects, and constraints) to admin users only. Faculty users can log in, view timetables, and manage their own constraints.

Special Constraints Module

The SpecialConstraints module allows both Admin and Faculty users to register scheduling constraints that are respected during timetable generation. Two constraint types are supported:

- Preferred Off: The faculty member should not be scheduled at a given day and time slot. The scheduling engine skips these slots when assigning lectures to this faculty member.
- Fixed Time: A specific lecture for a given class, division, and subject must be placed at a specific day and time slot. The engine applies all `fixed_time` constraints in Phase 0 before the main scheduling loop begins.

CSV Export

All four timetable views support CSV export using a zero-dependency JavaScript export library (excelExport.js). The generated CSV files include a UTF-8 BOM header for correct encoding in Excel on Windows. Each export is organized as an expanded grid with one row per session, spacer rows

between time slots, and a clear LUNCH BREAK label. The four export functions are: `exportClassTimetable`, `exportMasterPractical`, `exportFacultyTimetable`, and `exportLabTimetable`.

3.2 Advantages of the Proposed System

- Fully Automated End-to-End Pipeline: Eliminates all manual timetable preparation across practicals and lectures in minutes.
- Round-Robin Fair Scheduling: The $SY \rightarrow SY \rightarrow TY \rightarrow TY \rightarrow BE$ cycle ensures no year is locked out of available slots, replacing the rigid fixed-priority queue.
- Conflict-Free via In-Memory Constraint Caching: All constraints are loaded once at the start of generation, preventing repeated database queries and timeout errors.
- JWT Security with RBAC: Admin-only operations are protected at the API level with role-based decorators; faculty can only access their own data and constraints.
- Special Constraints Support: Both `preferred_off` and `fixed_time` constraints are respected during scheduling, giving institutional flexibility without manual intervention.
- CSV Export for All Views: Class, Faculty, Lab, and Master Practical timetables can all be downloaded as properly formatted CSV files compatible with Excel.
- Division-Specific Output: Generates individual timetables for each division (SY-A, SY-B, TY-A, BE-A, etc.) from a single generation run.
- Rollback Protection: The timetable generation endpoint snapshots existing data before running and rolls back on any failure, preventing partial or corrupt timetable states.

Chapter 4

WORKING ENVIRONMENT

4.1 Hardware Requirements

Component	Specification
Processor	Intel Core i3 or higher (i5/i7 recommended)
RAM	Minimum 4 GB (8 GB recommended)
Storage	Minimum 10 GB free disk space
Operating System	Windows 10/11, Ubuntu 20.04+, or macOS 11+
Network	Internet connection for cloud deployment and multi-user access
Browser	Google Chrome, Mozilla Firefox, or Microsoft Edge (latest version)

4.2 Software Requirements

Technology	Version/Type	Purpose
React.js + Vite	React 18, Vite 5	Frontend UI with authentication-aware routing, timetable visualization, constraints UI
TypeScript	v5.5+	Type-safe frontend development (tsconfig.app.json, tsconfig.node.json)
Tailwind CSS	v3.x	Utility-first CSS for responsive interface design
React Router DOM	v7.x	Client-side routing with protected routes and role-based navigation
Axios	v1.x	HTTP client with request interceptor for automatic JWT Bearer token attachment
Python (Flask)	Python 3.10+, Flask 2.x	Backend REST API, scheduling engine, JWT validation, role enforcement
PyJWT	v2.8+	JWT token generation (HS256) and verification for authentication
Werkzeug	Latest	Password hashing and verification for secure credential storage
MongoDB	v6.x (Atlas/Local)	NoSQL database for all application data and generated timetables
Flask-CORS	Latest	Cross-Origin Resource Sharing for React ↔ Flask communication

python-dotenv	Latest	Environment variable management for MONGO_URI, SECRET_KEY, DB_NAME
Greedy CSP + Round-Robin Engine	Custom (Phase II)	Core scheduling algorithm for practical and lecture timetable generation
excelExport.js	Custom	Zero-dependency CSV export library with UTF-8 BOM for all timetable views
Node.js	v18+	Runtime for frontend development and build tooling
Git	Latest	Version control and collaborative development

Chapter 5

METHODOLOGY

5.1 System Architecture

Schedulo follows a three-tier client-server architecture comprising the Presentation Layer (React.js + TypeScript frontend), the Application Layer (Python Flask backend with scheduling engine and JWT middleware), and the Data Layer (MongoDB database).

The frontend is built with React.js, Vite, and TypeScript, styled using Tailwind CSS. It implements protected routes using React Router DOM v7, with role-based navigation: Admin users access Generate Timetable, Faculty Data, Labs Data, Class Structure, and Special Constraints; Faculty users access View Timetables, Faculty Timetables, and Special Constraints. An AuthContext manages the JWT token and user state globally, while the Axios API instance automatically attaches the Bearer token to every request.

The Flask backend exposes RESTful API endpoints organized into modules: `auth_handler` (login/register/reset-password), `faculty_handler`, `labs_handler`, `class_structure_handler`, `subjects_handler`, `workload_handler`, `settings_handler`, `constraints_handler`, `timetable_generator`, `class_timetable_handler`, `lecture_tt_generator`, and `timetable_handler`. Two Flask decorators (`@token_required` and `@role_required`) enforce authentication and authorization on every protected route. The scheduling pipeline (`regenerate_master_practical_timetable`) includes snapshot-and-rollback protection.

MongoDB stores all master data and generated timetable documents in flexible JSON-based collections: `faculty`, `labs`, `class_structure`, `subjects`, `workload`, `settings`, `constraints`, `master_lab_timetable`, and `class_timetable`.

5.2 Work Flow

The complete workflow of Schedulo from login to timetable export is as follows:

1. Admin or Faculty logs in via the Schedulo Login page. Credentials are submitted to POST `/api/auth/authenticate`. The backend validates the password hash and issues a signed JWT token.
2. Admin enters Master Data: Faculty records (with email for welcome email via SendGrid), Lab information with session timings, and Class Structure (divisions and batches per year).
3. Admin configures Department Timings (day start/end, lecture duration, breaks). The backend calculates available time slots dynamically and saves them to MongoDB.
4. Admin enters Subjects per year and Faculty Assignments (workload) per division. Duplicate workload entries are rejected at the API level.
5. (Optional) Admin or Faculty registers Special Constraints: preferred_off time slots or fixed_time lecture pins for specific class-division-subject combinations.
6. Admin triggers timetable generation. The Flask backend snapshots existing data, then executes the three-step pipeline: Practical Generator → Class Timetable Handler → Lecture Generator. On failure, the snapshot is restored.
7. Step 1 (Practical Generator): Constraints loaded into memory. Round-Robin CSP assigns practical slots to batches. Output: Master Lab Timetable.
8. Step 2 (Class Timetable Handler): Data Mapper converts lab-wise data to class-wise format with batch integer normalization. Locked Practical slots written per division.
9. Step 3 (Lecture Generator): Constraints loaded into memory. Fixed-time constraints applied first (Phase 0). Smart Queue with Round-Robin fills remaining slots using Tier 1 (Morning) and Tier 2 (Afternoon fallback). Spread constraint prevents same subject twice in one day per class.
10. Generated timetables saved to MongoDB and displayed on the frontend: Class Timetables (by division), Faculty Timetables (by faculty name), Lab Timetables (by lab name), and Master Practical Plan (lab-wise across all years).
11. Admin and Faculty export any timetable view as a CSV file using the Download CSV button.

Chapter 6

FLOW DIAGRAMS

6.1 Data Flow Diagram

The Data Flow Diagram (DFD) illustrates how data moves through the Schedulo system from initial data entry by the administrator to the final timetable output.

Level 0 – Context Diagram:

- Input: Admin enters Faculty Data, Labs Data, Class Structure, Department Timings, Subjects, Faculty Assignments, and Special Constraints. Faculty users enter preferred-off or fixed-time constraints.
- Process: Schedulo System authenticates users via JWT, processes all inputs through the three-step scheduling pipeline, and enforces role-based access at every endpoint.
- Output: Division-wise Class Timetable, Faculty-wise Timetable, Lab Timetable, and Master Practical Plan delivered to Admin and Faculty. All outputs available as CSV downloads.

Level 1 DFD – Key Processes:

- Process 1.0 – Authentication & Authorization: Validates JWT tokens on all protected routes. Restricts admin-only operations (generate, manage data) via `@role_required` decorator.
- Process 2.0 – Data Management: Receives and validates master data (faculty, labs, class structure) and configuration data (timings, subjects, assignments, constraints) and stores in MongoDB. Duplicate detection enforced.
- Process 3.0 – Practical Generator: Reads workload and faculty data; loads constraints into memory; applies Greedy CSP with Round-Robin cycle; outputs Master Lab Timetable.
- Process 4.0 – Class Timetable Handler: Reads Master Lab Timetable; normalizes batch values; applies Data Mapper; outputs Locked Practical slots per division.
- Process 5.0 – Lecture Generator: Reads locked slots and remaining faculty availability; loads constraints into memory; applies fixed-time constraints first; fills remaining slots with Smart Queue Round-Robin; enforces spread constraint.

- Process 6.0 – Export Module: Client-side CSV generation from displayed timetable data with UTF-8 BOM for Excel compatibility.

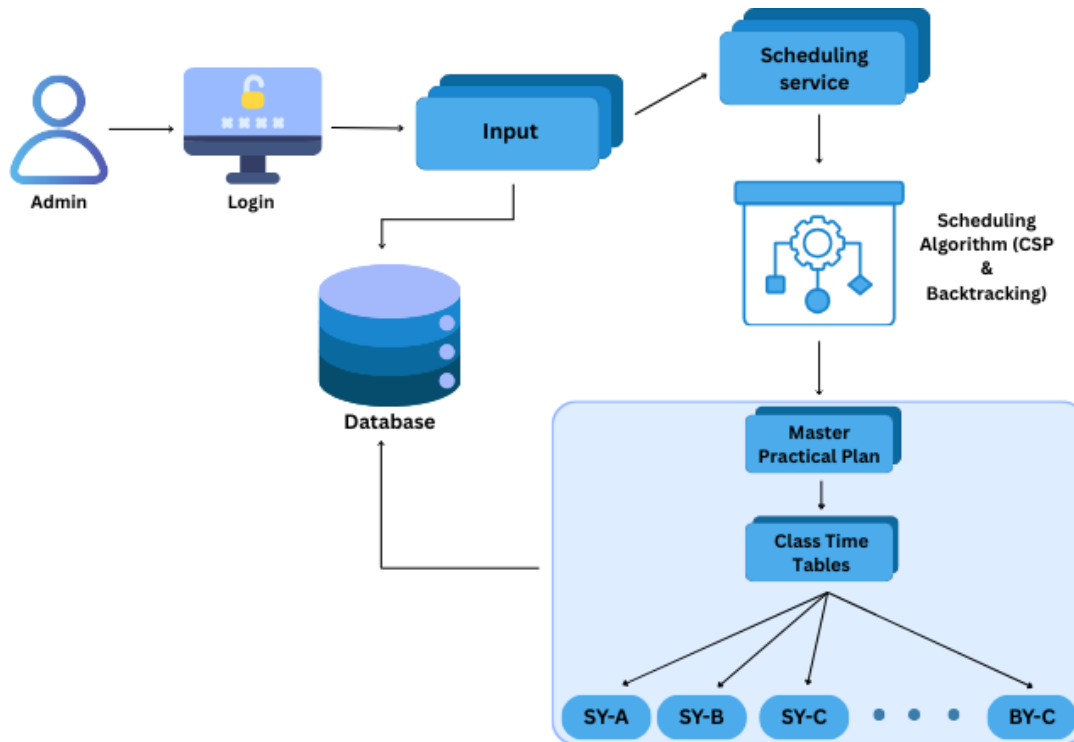


Fig. 3: Data Flow Diagram

6.2 Sequential UML Diagram

The Use Case Diagram of the Schedulo system illustrates the interaction between two main actors: Admin and Faculty.

Admin Use Cases:

- Login / Register / Reset Password
- Enter Faculty Data (with email for welcome notification)
- Enter Labs Data and configure Lab Session Timings
- Define Class Structure (divisions and batches per year)
- Configure Department Timings (lecture duration, day start/end, breaks)

- Enter Subjects per year (with practical configuration and required lab)
- Assign Faculty to Subjects (workload assignment per division)
- Add / Delete Special Constraints for any faculty
- Generate Timetable (triggers full three-step pipeline with rollback protection)
- View Master Practical Plan, Class Timetables, Faculty Timetables, Lab Timetables
- Export any timetable view as CSV

Faculty Use Cases:

- Login / Reset Password
- View Class Timetable (own division)
- View Personal Faculty Timetable (assigned lectures and practicals)
- Add / Delete Own Special Constraints (preferred_off or fixed_time)
- Export own timetable as CSV

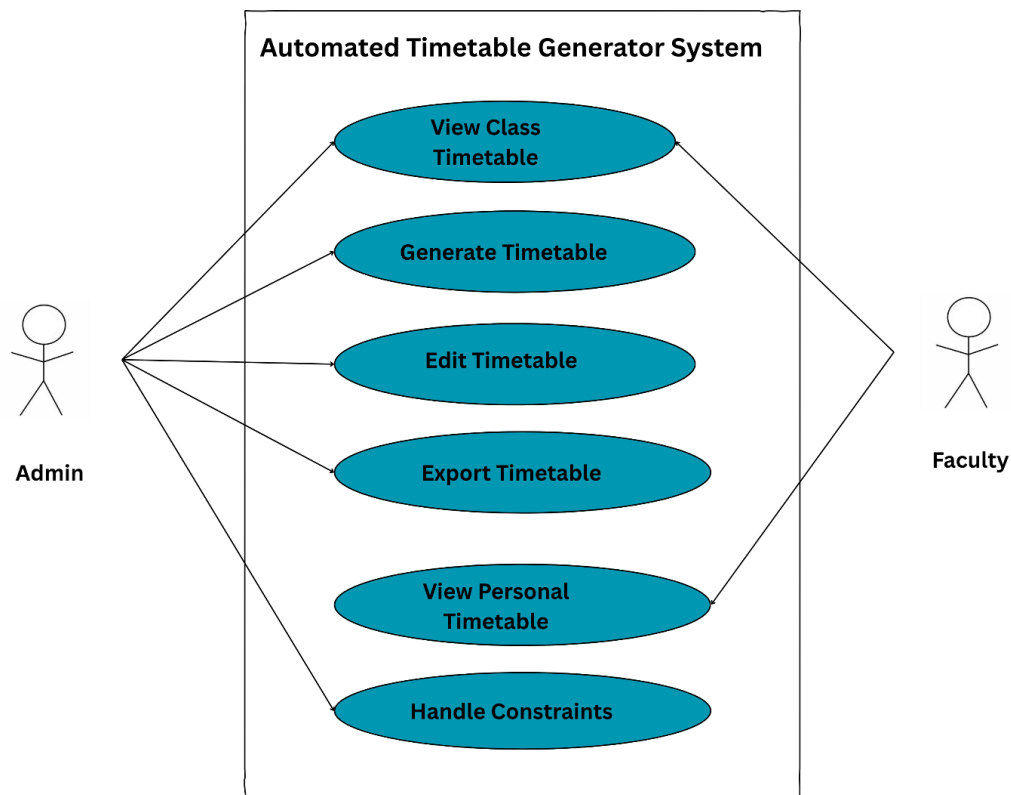


Fig. 4: Use Case Diagram

Chapter 7

IMPLEMENTATION

7.1 Code Snippet

The following code snippets illustrate the key new components implemented in Schedulo Phase II. The backend is organized under the modules/ directory.

1. JWT Authentication Decorator (app.py):

```
def token_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        token = request.headers.get('Authorization', '').split(' ')[1]
        data = jwt.decode(token, SECRET_KEY, algorithms=['HS256'])
        request.user = data
        return f(*args, **kwargs)
    return decorated

def role_required(role):
    def wrapper(f):
        @wraps(f)
        def decorated(*args, **kwargs):
            if request.user.get('role') != role:
                return jsonify({'error': 'Forbidden'}), 403
            return f(*args, **kwargs)
        return decorated
    return wrapper
```

2. Round-Robin Scheduling Order (timetable_generator.py & lecture_tt_generator.py):

```
ROUND_ROBIN_CYCLE = ['SY', 'SY', 'TY', 'TY', 'BE'] # repeating

def _build_round_robin_order(pending_keys):
    by_year = {}
    for k in pending_keys:
        by_year.setdefault(k[0], []).append(k)
    # Walk cycle pattern, picking next key from each year group
    # Guarantees BE gets 1 turn per 5 cycles, SY/TY get 2 turns each
    # No year is ever completely locked out of slot assignment
```

3. In-Memory Constraint Caching (constraints_handler.py):

```
def _load_constraints(self):
```

```

# Load ALL constraints ONCE at generation start
# Prevents thousands of MongoDB queries during hot loop
for c in constraints_collection.find({}):
    fac = c.get('faculty_short', c.get('faculty_name', ''))
    if c.get('type') == 'preferred_off':
        self.preferred_off.add((fac, c['day'], c['time_slot']))
    elif c.get('type') == 'fixed_time':
        self.fixed_time_locked.add((fac, c['day'], c['time_slot']))

```

4. CSV Export with UTF-8 BOM (excelExport.js):

```

function downloadCSV(csvString, filename) {
    const BOM = '\uFEFF'; // UTF-8 BOM for correct Excel encoding on Windows
    const blob = new Blob([BOM + csvString],
        { type: 'text/csv;charset=utf-8;' });
    const url = URL.createObjectURL(blob);
    // trigger download...
}

```

5. Timetable Generation with Rollback Protection (app.py):

```

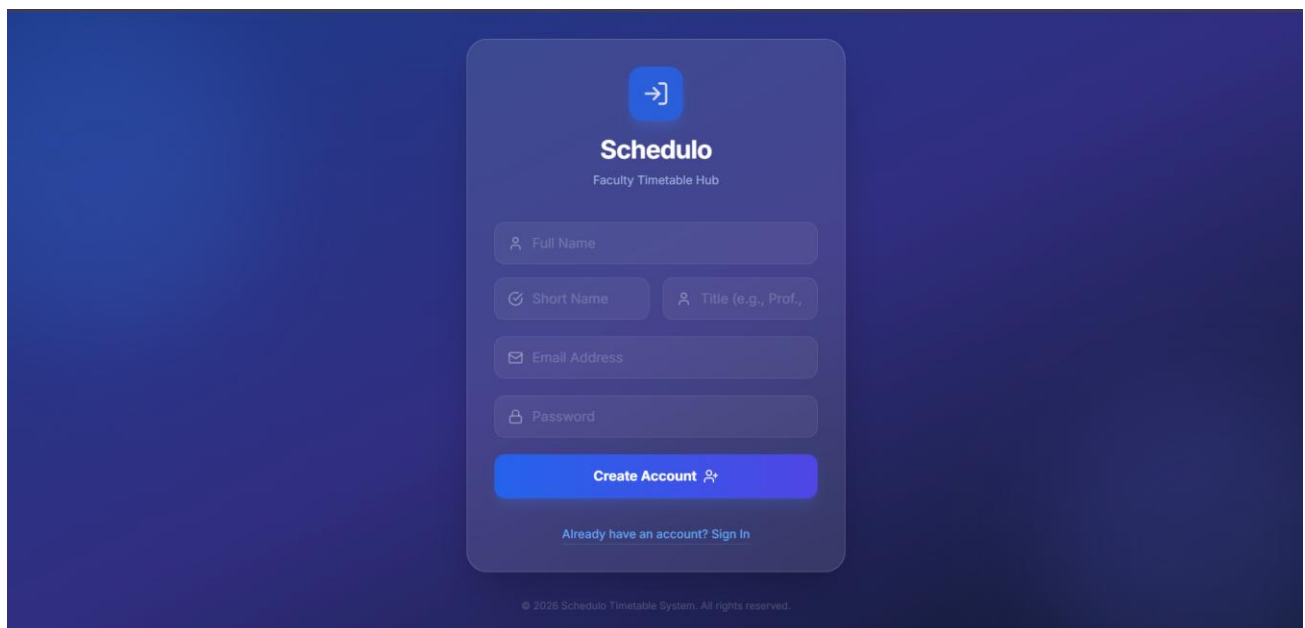
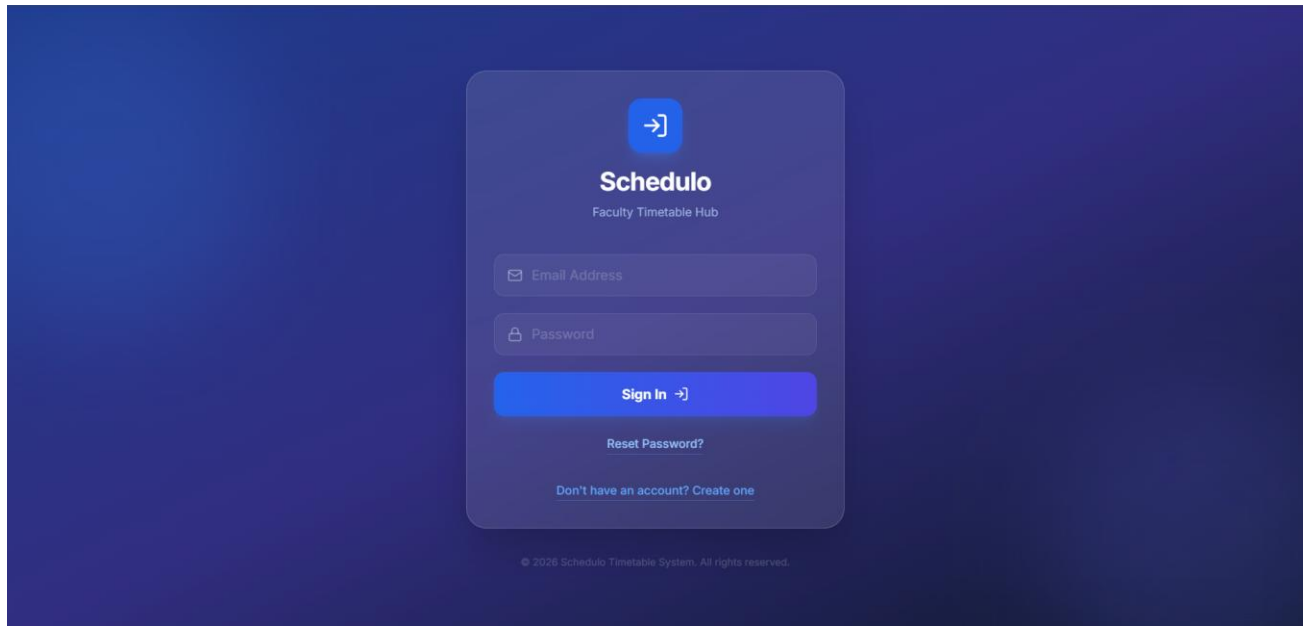
@app.route('/api/regenerate_master_practical_timetable', methods=['POST'])
@token_required
@role_required('admin')
def regenerate_master_practical_timetable():
    snapshot_master = list(master_col.find({}))
    snapshot_class = list(class_col.find({}))
    def _rollback(reason):
        master_col.delete_many({})
        class_col.delete_many({})
        master_col.insert_many(snapshot_master)
        class_col.insert_many(snapshot_class)
    # Run pipeline; call _rollback() on any failure

```

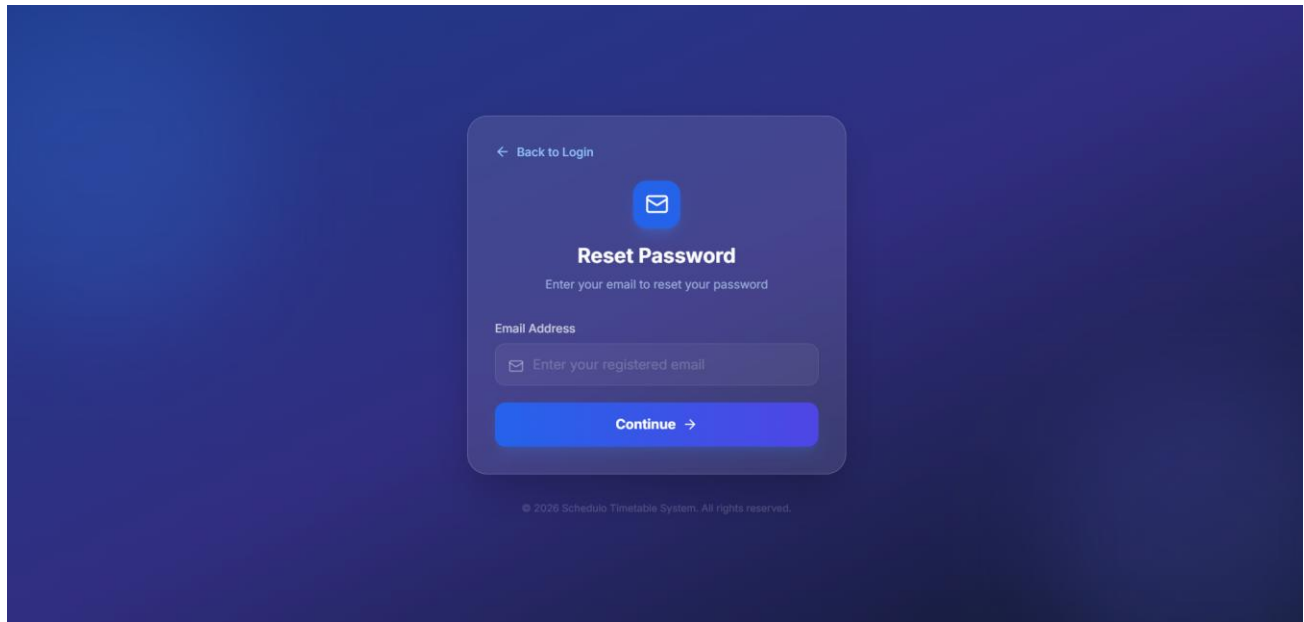
7.2 Screenshots & Results

The following screenshots demonstrate the working Schedulo system across its key functional modules. All outputs include actual generated data from WIT Solapur CSE Department test runs.

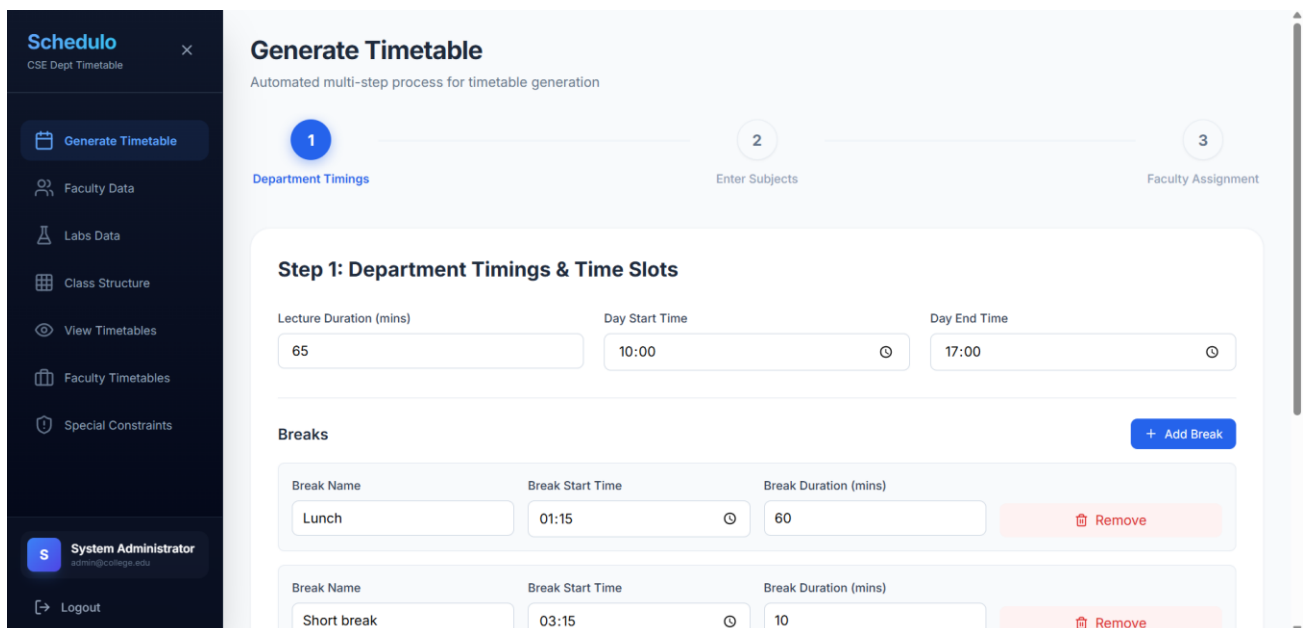
- Login Page: Email/password login form with glassmorphism design. Auto-switches to register mode if faculty not found.



- Reset Password Page: Two-flow password reset: unauthenticated (email verification) and authenticated (current password required).



- Admin Dashboard / Sidebar: Role-aware navigation showing Generate Timetable, Faculty Data, Labs Data, Class Structure, View Timetables, Faculty Timetables, Special Constraints.



- Faculty Data Page: Table of all faculty with title, name, short name. Add/Edit/Delete with email field. Welcome email sent via SendGrid on faculty creation.

Schedulo
CSE Dept Timetable

Faculty Data
Manage faculty information

+ Add Faculty

Title	Full Name	Short Name	Actions
	System Administrator	ADMIN	Edit Delete
Prof	Rajesh Sharma	RS	Edit Delete
Prof	Anil Kumar	AK	Edit Delete
Prof	Nita Patel	NP	Edit Delete
Prof	Harpreet Sign	HS	Edit Delete
Prof	Sunita Gupta	SG	Edit Delete
Prof	Vijay Mehta	VM	Edit Delete
Prof	Priya Joshi	PJ	Edit Delete
Prof	Daksh Varma	DV	Edit Delete

System Administrator
admin@college.edu

Logout

- Labs Data Page: Split layout with Lab Session Timings sidebar and Labs table. Add/Edit/Delete labs and define global practical session times.

Schedulo
CSE Dept Timetable

Labs Management
Manage academic laboratories and session timings

+ Add New Lab

Lab Slots
Global Timings

SLOT 1
11:15 — 13:15

SLOT 2
14:15 — 16:15

SLOT 3
16:20 — 18:20

Lab Name	Short Form	Actions
Network Lab	NL	Edit Delete
Programming lab	PL	Edit Delete
Database Lab	DL	Edit Delete
AI Lab	AL	Edit Delete
Electronics Lab	EL	Edit Delete

System Administrator
admin@college.edu

Logout

- Class Structure Page: Three-card layout for SY, TY, and BE (Final Year) showing num_divisions and batches_per_division inputs.

The screenshot shows the 'Class Structure' configuration page. The left sidebar contains navigation links: 'Generate Timetable', 'Faculty Data', 'Labs Data', 'Class Structure' (selected), 'View Timetables', 'Faculty Timetables', 'Special Constraints', 'System Administrator', and 'Logout'. The main content area is titled 'Class Structure' with the subtitle 'Configure divisions and batches for each year'. It features three cards for 'Final year (BE)', 'Second year (SY)', and 'Third year (TY)'. Each card has input fields for 'Number of Divisions' and 'Batches per Division', and a 'Total Batches' summary. The 'Final year (BE)' card shows 1 division and 3 batches per division, totaling 3 batches. The 'Second year (SY)' card shows 2 divisions and 3 batches per division, totaling 6 batches. The 'Third year (TY)' card shows 2 divisions and 3 batches per division, totaling 6 batches. A 'Save Class Structure' button is located at the bottom right of the main content area.

- Department Timings (Step 1): Lecture duration, day start/end time inputs, repeatable breaks configuration with Add/Remove.

The screenshot shows the 'Step 1: Department Timings & Time Slots' configuration page. The left sidebar is identical to the previous screenshot. The main content area is titled 'Step 1: Department Timings & Time Slots'. It has input fields for 'Lecture Duration (mins)' (65), 'Day Start Time' (10:00), and 'Day End Time' (17:00). Below these is a 'Breaks' section with a '+ Add Break' button. The 'Breaks' section contains three rows of break configuration, each with 'Break Name', 'Break Start Time', 'Break Duration (mins)', and a 'Remove' button. The first row is 'Lunch' at 01:15 for 60 minutes. The second row is 'Short break' at 03:15 for 10 minutes. The third row is 'kjsksj' at 10:01 for 20 minutes. A large blue button at the bottom of the main content area is labeled 'Set/Update Timings & Generate Slots'.

- Subject Entry (Step 2): Per-year subject table with full name, short name, lectures/week, practicals/week, practical duration, practical type (Specific Lab / Common Lab), and required lab dropdown.

Schedulo
CSE Dept Timetable

Generate Timetable

Faculty Data

Labs Data

Class Structure

View Timetables

Faculty Timetables

Special Constraints

S System Administrator
admin@college.edu

Logout

Step 2: Subjects for CSE

Add and manage subjects for each year

Select Year
2nd Year (SY)

+ Add Subject

Subjects for 2nd Year (SY) (5)

Subject Name	Short Name	Lectures/Week	Practicals/Week	Practical Duration	Practical Type
Data Structures	DS	4	1	2 slots	Common Lab
Digital Electronics	DE	2	1	2 slots	Common Lab
Discreet Mathematics	DM	4	0		

- Faculty Assignment (Step 3): Per-year workload assignment table showing faculty, subject, division, batches, theory/practical hrs/week. Conditional batch selector appears only for practical subjects.

Schedulo
CSE Dept Timetable

Generate Timetable

Faculty Data

Labs Data

Class Structure

View Timetables

Faculty Timetables

Special Constraints

S System Administrator
admin@college.edu

Logout

Step 3: Faculty Workload Assignment

Assign faculties to subjects and manage their workload for each year

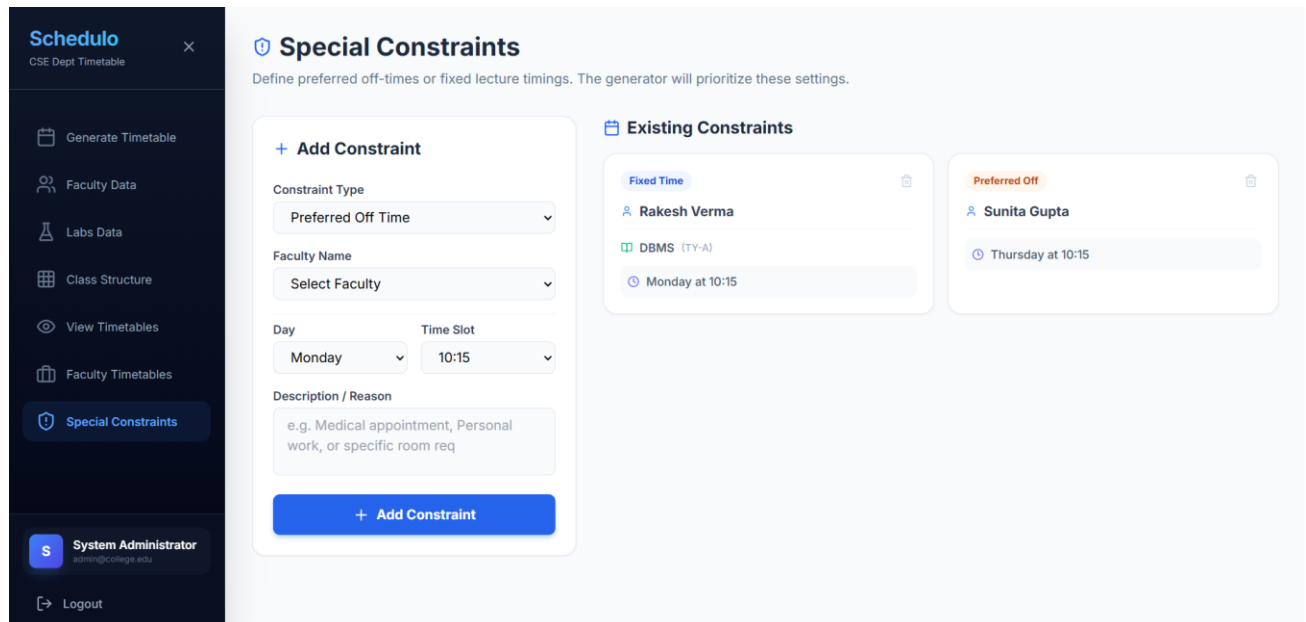
Select Year
2nd Year (SY)

+ Add Assignment

Faculty Workload for 2nd Year (SY) (10)

FACULTY	SUBJECT	DIVISION & BATCHES	WORKLOAD	ASSIGNMENT TYPE
Prof Anil Kumar	Data Structures DS	Division A - 1, 2, 3	4 (Theory) + 2 (Practical) hrs/week	Theory + Practical
Prof Nita Patel	Object Oriented Programming OOPS	Division A - 1, 2, 3	3 (Theory) + 2 (Practical) hrs/week	

- Special Constraints Page: Two-column layout with Add Constraint form (type, faculty, day, time slot, optional year/division/subject for fixed_time) and existing constraints grid.



Schedule
CSE Dept Timetable

Special Constraints
Define preferred off-times or fixed lecture timings. The generator will prioritize these settings.

+ Add Constraint

Constraint Type: Preferred Off Time

Faculty Name: Select Faculty

Day: Monday Time Slot: 10:15

Description / Reason: e.g. Medical appointment, Personal work, or specific room req

+ Add Constraint

Existing Constraints

Fixed Time

Rakesh Verma

DBMS (TY-A)

Monday at 10:15

Preferred Off

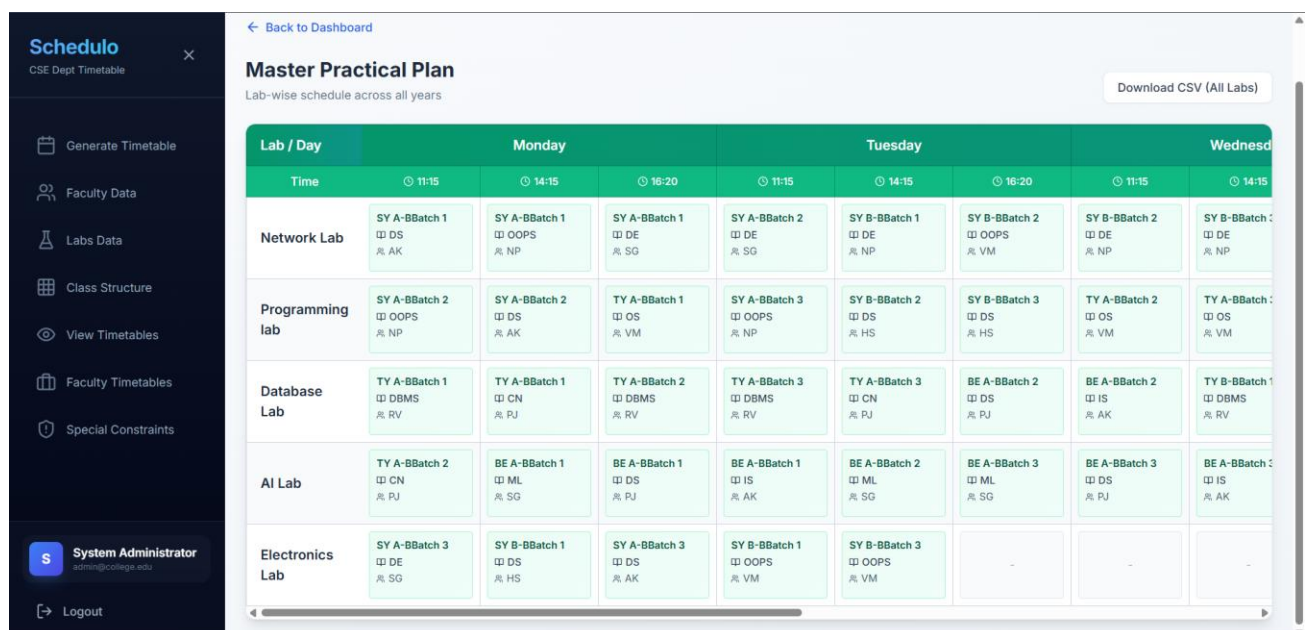
Sunita Gupta

Thursday at 10:15

System Administrator
admin@college.edu

Logout

- Master Practical Plan View: Lab-wise grid across all years. Download CSV (All Labs) button exports all lab timetables as separate CSV files.



Schedule
CSE Dept Timetable

Master Practical Plan
Lab-wise schedule across all years

[Download CSV \(All Labs\)](#)

Lab / Day	Monday			Tuesday			Wednesday	
Time	11:15	14:15	16:20	11:15	14:15	16:20	11:15	14:15
Network Lab	SY A-BBatch 1 DS AK	SY A-BBatch 1 OOPS NP	SY A-BBatch 1 DE SG	SY A-BBatch 2 DE SG	SY B-BBatch 1 DE NP	SY B-BBatch 2 OOPS VM	SY B-BBatch 2 DE NP	SY B-BBatch 2 DE NP
Programming lab	SY A-BBatch 2 OOPS NP	SY A-BBatch 2 DS AK	TY A-BBatch 1 OS VM	SY A-BBatch 3 OOPS NP	SY B-BBatch 2 DS HS	SY B-BBatch 3 DS HS	TY A-BBatch 2 OS VM	TY A-BBatch 2 OS VM
Database Lab	TY A-BBatch 1 DBMS RV	TY A-BBatch 1 CN PJ	TY A-BBatch 2 DBMS RV	TY A-BBatch 3 DBMS RV	TY A-BBatch 3 CN PJ	BE A-BBatch 2 DS PJ	BE A-BBatch 2 IS AK	TY B-BBatch 1 DBMS RV
AI Lab	TY A-BBatch 2 CN PJ	BE A-BBatch 1 ML SG	BE A-BBatch 1 DS PJ	BE A-BBatch 1 IS AK	BE A-BBatch 2 ML SG	BE A-BBatch 3 ML SG	BE A-BBatch 3 DS PJ	BE A-BBatch 2 IS AK
Electronics Lab	SY A-BBatch 3 DE SG	SY B-BBatch 1 DS HS	SY A-BBatch 3 DS AK	SY B-BBatch 1 OOPS VM	SY B-BBatch 3 OOPS VM	-	-	-

System Administrator
admin@college.edu

Logout

- Class Timetable View (SY-A Division): Weekly grid with colour-coded lecture (blue) and practical (green) cells. Each cell shows subject, faculty, batch, and lab. Download CSV button.

Schedulo
CSE Dept Timetable

- Generate Timetable
- Faculty Data
- Labs Data
- Class Structure
- View Timetables
- Faculty Timetables
- Special Constraints

S System Administrator
admin@college.edu

Logout

SY-A — Division A
Generated on 04 May 2026, 12:18 pm - 9 practicals assigned

Download CSV

Time / Day	Monday	Tuesday	Wednesday	Thursday	Friday
10:15	DM — Discreet Mathematics RS	DM — Discreet Mathematics RS	CO — Computer Organisation SG	DM — Discreet Mathematics RS	—
11:15	1 DS — Data Structures AK Network Lab 2 OOPS — Object Oriented Programming NP Programming lab 3 DE — Digital Electronics SG Electronics Lab	2 DE — Digital Electronics SG Network Lab 3 OOPS — Object Oriented Programming NP Programming lab	DM — Discreet Mathematics RS	DS — Data Structures AK	—
12:15	—	DS — Data Structures AK	DS — Data Structures AK	OOPS — Object Oriented Programming NP	—
13:15	—	—	—	—	—

- Faculty Timetable View: Searchable faculty directory cards. Click to view individual faculty weekly schedule. Download CSV button.

Schedulo
CSE Dept Timetable

- Generate Timetable
- Faculty Data
- Labs Data
- Class Structure
- View Timetables
- Faculty Timetables
- Special Constraints

S System Administrator
admin@college.edu

Logout

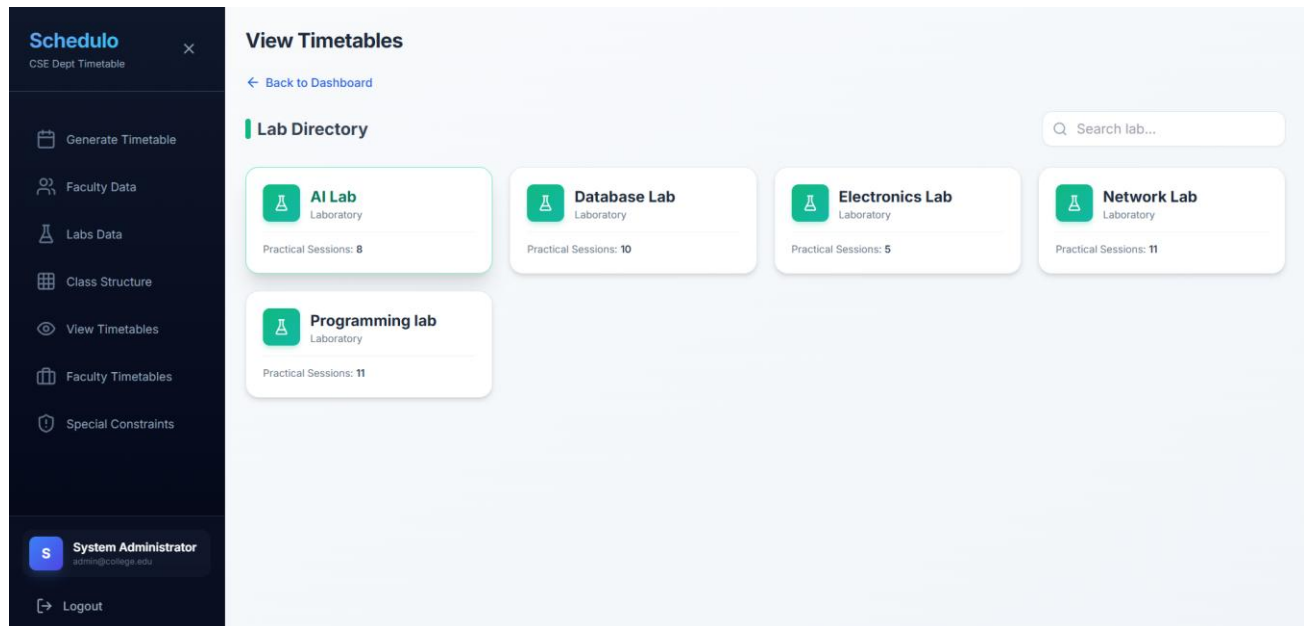
View Timetables
[Back to Dashboard](#)

Faculty Directory

Search faculty...

AK Prof Anil Kumar Theory Sessions: 5 Practical Sessions: 6	HS Prof Harpreet Sign Theory Sessions: 9 Practical Sessions: 3	MD Prof Meera Desai Theory Sessions: 9 Practical Sessions: 0	NP Prof Nita Patel Theory Sessions: 5 Practical Sessions: 6
PJ Prof Priya Joshi Theory Sessions: 10 Practical Sessions: 9	RS Prof Rajesh Sharma Theory Sessions: 8 Practical Sessions: 0	RV Prof Rakesh Verma Theory Sessions: 8 Practical Sessions: 6	SG Prof Sunita Gupta Theory Sessions: 15 Practical Sessions: 6
VM Prof Vijay Mehta Theory Sessions: 10 Practical Sessions: 9			

- Lab Timetable View: Searchable lab directory cards showing practical session count. Click to view lab-wise weekly schedule. Download CSV button.



- Generated CSV Sample: SY-A timetable CSV with UTF-8 BOM, expanded rows per batch, spacer rows between time slots, and LUNCH BREAK label at 13:15.

Time / Day						
	A	B	C	D	E	F
1	Time / Day	Monday	Tuesday	Wednesday	Thursday	Friday
2		DM (Discreet Mathematics) 10:15 Faculty: RS	DM (Discreet Mathematics) Faculty: RS	CO (Computer Organisation) Faculty: SG	DM (Discreet Mathematics) Faculty: RS	
3						
4		DS (Data Structures) Faculty: AK Batch: 1 11:15 Lab: Network Lab	DE (Digital Electronics) Faculty: SG Batch: 2 Lab: Network Lab	DM (Discreet Mathematics) Faculty: RS	DS (Data Structures) Faculty: AK	
		OOPS (Object Oriented Programming) Faculty: NP Batch: 2 Lab: Programming Lab	OOPS (Object Oriented Programming) Faculty: NP Batch: 3 Lab: Programming Lab			

Chapter 8

TESTING REPORT

The Schedulo system was tested using unit testing, integration testing, and system-level validation against real departmental data from WIT Solapur CSE Department. New Phase II tests cover authentication, constraints, and CSV export functionality.

#	Test Case	Input	Expected Output	Status
1	JWT login with valid credentials	Admin email + correct password	Token issued, role=admin in payload	✓ Pass
2	JWT login with invalid password	Admin email + wrong password	HTTP 401 Unauthorized	✓ Pass
3	Role-based access: faculty tries admin route	Faculty JWT + POST /regenerate	HTTP 403 Forbidden	✓ Pass
4	Round-Robin practical scheduling (3 years)	SY, TY, BE batches mixed queue	Slots distributed fairly: SY→SY→TY→TY→BE cycle	✓ Pass
5	Faculty double-booking prevention	Same faculty, 2 subjects, same slot	Second subject moved to next available slot	✓ Pass
6	Lab double-booking prevention	Two batches, same lab, same slot	Second batch assigned to alternate lab	✓ Pass
7	preferred_off constraint respected	Faculty has preferred_off Monday 11:15	No lecture placed for that faculty at that slot	✓ Pass
8	fixed_time constraint applied (Phase 0)	Admin pins SY-A DS lecture to Tuesday 10:15	DS placed at Tuesday 10:15 before main loop	✓ Pass
9	In-memory constraint caching	Generation with 10 constraints, 50 slots	No DB query per slot; generation completes < 5s	✓ Pass
10	Spread constraint enforcement	Same subject queued for Monday twice	Second occurrence placed on different day	✓ Pass
11	Rollback on pipeline failure	Corrupt workload data triggers Step 2 failure	Previous timetable restored from snapshot	✓ Pass
12	CSV export – Class Timetable	SY-A timetable, Download CSV clicked	UTF-8 BOM CSV file downloaded with correct slot data	✓ Pass

13	CSV export – Faculty Timetable	Faculty 'AMP' timetable exported	CSV shows all theory + practical sessions correctly	✓ Pass
14	Duplicate workload entry rejection	Same faculty, year, division, subject submitted twice	HTTP 409 Conflict returned, no duplicate inserted	✓ Pass
15	Full pipeline end-to-end test	Complete WIT CSE SY+TY+BE data	All divisions scheduled, all CSV exports functional	✓ Pass

Chapter 9

CONCLUSION

The **Schedulo – Automated Timetable Generator** has successfully evolved into a complete, secure, and end-to-end academic scheduling platform. In Phase II, the system moved beyond its Phase I Brute Force foundation to implement a production-ready solution comprising four major new components:

- A three-stage scheduling pipeline: Greedy CSP Practical Generator with Round-Robin fairness, Class Timetable Handler with Data Mapping, and Smart Queue Lecture Generator with in-memory constraint caching.
- JWT-based Authentication with Role-Based Access Control separating Admin and Faculty capabilities at the API level.
- A Special Constraints module supporting `preferred_off` and `fixed_time` constraint types, applied before and during scheduling.
- CSV Export functionality for all four timetable views with proper UTF-8 encoding for cross-platform compatibility.

The Round-Robin scheduling cycle ($SY \rightarrow SY \rightarrow TY \rightarrow TY \rightarrow BE$) replaced the previous fixed priority queue, ensuring all academic years receive fair access to available time slots. In-memory constraint caching eliminated the per-slot database query bottleneck that caused generation timeouts in earlier iterations. Rollback protection ensures that failed generation attempts never corrupt the existing timetable state.

Overall, Schedulo Phase II demonstrates that a well-designed constraint-based scheduling pipeline, combined with proper authentication, flexible constraint management, and structured export capabilities, can effectively solve the academic timetabling problem for real engineering institutions. The system is ready for departmental deployment and establishes a strong technical foundation for future optimization enhancements.

Chapter 10

FUTURE WORK

1. Meta-Heuristic Optimization Algorithms

With the core scheduling pipeline complete, the next phase will integrate Simulated Annealing (SA), Tabu Search (TS), and Genetic Algorithm (GA) as post-processing optimization layers. These algorithms will evaluate generated timetables and iteratively improve slot distribution, faculty workload balance, and resource utilization without violating hard constraints.

2. Google OR-Tools Integration

Google OR-Tools will be evaluated for integration to solve large-scale scheduling instances more efficiently than the current greedy approach, particularly for institutions with a large number of divisions, batches, and subjects.

3. Dynamic Rescheduling

A dynamic rescheduling feature will be developed to adjust timetables in real time when changes occur, such as faculty unavailability, lab maintenance, or new subject additions. This will make the system more responsive to practical institutional situations.

4. Elective Subject and Cross-Departmental Faculty Support

Support for elective subjects shared across divisions and for faculty members shared across multiple departments will be introduced, expanding the system's applicability beyond single-department use.

5. Cloud Deployment and Multi-User Access

The system will be deployed on a cloud platform (AWS or Google Cloud) for multi-user access, real-time data synchronization across departments, and institutional-scale testing and validation.

6. Timetable Quality Analytics Dashboard

A reporting dashboard will be developed to analyze timetable quality metrics such as faculty workload distribution, subject spread across the week, lab utilization rate, and gap detection, helping administrators make data-driven scheduling decisions.

7. XLSX Export Support

While CSV export is currently implemented, future work will add native .xlsx file export with formatted tables, color-coded session types (lecture vs. practical), and merged cells for improved readability in Microsoft Excel.

Chapter 11

REFERENCES

- [1] ASC Timetables. (n.d.). ASC timetable scheduling software. Retrieved from <https://www.asctimetables.com/>
- [2] TimetableMaster. (n.d.). Timetable scheduling for schools. Retrieved from <https://www.timetablemaster.com/>
- [3] Timetabler. (n.d.). Timetabler school scheduling software. Retrieved from <https://www.timetabler.com/>
- [4] Alonge, O., & Sakpere, W. (2024). Timetable scheduler using NSGA-II for optimized scheduling in educational institutions. *Journal of Educational Technology Systems*.
- [5] Farinola, L., & Assogba, M. (2025). Explicit artificial intelligence timetable generator for colleges and universities. *Open Journal of Applied Sciences*.
- [6] Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4th ed.). Pearson.
- [7] Colomi, A., Dorigo, M., & Maniezzo, V. (1992). Genetic algorithms and the timetabling problem. In *Proceedings of the First Workshop on Parallel Problem Solving from Nature* (pp. 439–446). Springer.
- [8] Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680.
- [9] Glover, F. (1989). Tabu search – Part I. *ORSA Journal on Computing*, 1(3), 190–206.
- [10] Google. (n.d.). OR-Tools: Constraint optimization. Google Developers. Retrieved from <https://developers.google.com/optimization>
- [11] OWASP. (n.d.). JSON Web Token (JWT) security cheat sheet. Retrieved from https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html
- [12] MongoDB, Inc. (n.d.). MongoDB documentation. Retrieved from <https://www.mongodb.com/docs/>
- [13] Python Software Foundation. (n.d.). Python 3 documentation. Retrieved from <https://docs.python.org/3/>
- [14] Pallets Projects. (n.d.). Flask documentation. Retrieved from <https://flask.palletsprojects.com/>
- [15] Meta Open Source. (n.d.). React documentation. Retrieved from <https://react.dev/>
- [16] Axios. (n.d.). Axios HTTP client documentation. Retrieved from <https://axios-http.com/docs/intro>
- [17] Vitejs. (n.d.). Vite documentation. Retrieved from <https://vitejs.dev/>